

exp-fips203-mlkem-pke-keygen-k2

FIPS 203 Alg.13 ML-KEM-512 PKE KeyGen 实现方案

ascendc 工程 (ML-KEM-512 W4a / E13)

2026-07-27

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-keygen-k2/`: 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 13**, 即 ML-KEM-512 ($k=2$) **PKE KeyGen**。

参数锁定: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem512-parameter-card.md` (S-1 / T-B2) 与活跃探针 `ascendc-tests/ml-kem/ml-kem-512/pass-fix-f203-alg13-device-keygen-k2/STATUS.md`。**生产 I/O**: `input/` 仅 `seed_d.bin` 与 NTT LUT; `output/` 仅 `ek_pke.bin` (800B) 与 `dk_pke.bin` (768B)。**计划 AI Core 数**: 2 launch; Launch 1 为 AIV_ONLY blockDim=2, Launch 2 为 MIX blockDim=1 (1 AIC + 2 AIV, 单 cube)。**禁止**: 写入 `examples/stable/ml-kem/ml-kem-512/`; 从任何 `frozen/` 目录复制、改写或移植源码; 用 k3/k4 的 ek/dk 长度或零垫到 3/4/6/8 替代 k2 几何。

目录

1 工程边界	2
1.1 允许的代码来源与依赖	2
1.2 禁止项	2
2 数学语义 (Alg.13, $k=2$, $n=256$, $q=3329$)	2
2.1 输入输出	2
2.2 阶段公式	2
3 AscendC 实现规划	3
3.1 Launch 与 AI Core 规模	3
3.2 GM 几何	3
3.3 同步与流水	3
4 AscendC API 表	4
4.1 评估过但未采用	5
5 数学步骤覆盖率	5

目录	2
6 验证计划	5
6.1 默认验收	5
6.2 可选对照	6
7 产出物	6

1 工程边界

1.1 允许的代码来源与依赖

来源	用途
ascendc-tests/ml-kem/m l-kem-512/pass-fix-f20 3-alg13-device-keygen- k2/	活跃 D13 k2 行为基线；可复制后自包含适配到本 exp, 最终不得保留对探针目录的编译期 include/import
examples/incubating/ml -kem/ml-kem-768/exp-fi ps203-mlkem-pke-keygen -k3/	活跃 k3 incubating 结构参考；仅 retarget 到 k2 锁 定几何与字节长
library/shared/	SHAKE/Keccak、CBD- $\eta = 3$ 、公共设备侧头文件 等共享库
CANN / tikicpulib / CAModel	编译、CPU 孪生与 SIM 运行环境

1.2 禁止项

- 禁止从 ascendc-tests/frozen/ 或 examples/frozen/ 拿源码、customspec 或路线。
- 禁止将 a_{hat} 、 s_{hat} 、 t_{hat} 等中间态作为默认 I/O 落盘；debug dump 只能显式开关启用。
- 禁止把 k3/k4 的 ek/dk 长度、polyvec6/8 或零垫当作默认。
- 禁止新建或写入 stable-512；本目录只作为 incubating 预研实现。

2 数学语义 (Alg.13, $k = 2$, $n = 256$, $q = 3329$)

2.1 输入输出

张量 / 文件	契约
input/seed_d.bin	4B little-endian uint32, 由 Host 派生 FIPS 203 的 d ; 域分离串为 <code>exp-mlkem-f203-2s1e-k2:SEED_D=</code>
input/lut_even_stacked.bin lut_odd_stacked.bin	NTT Stage2 LUT; 由本目录 golden 脚本生成；不是用户可见交付输入
output/ek_pke.bin	800B, $\text{ByteEncode}_{12}(\hat{t})$ 的 768B 后拼接 ρ 的 32B
output/dk_pke.bin	768B, $\text{ByteEncode}_{12}(\hat{s})$, 其中 $\hat{s}$ 为前 2 个秘密多项式

2.2 阶段公式

1. $G(d||\text{byte}(2)) \rightarrow (\rho, \sigma)$; k 字节锁定为 2。

2. 行 3-7: 对 $(i, j) \in \{0, 1\}^2$ 执行 **SampleNTT**, 生成 $\hat{A} \in R_q^{2 \times 2}$, GM 形状锁定为 [4,256] int32。
3. 行 8-15: 用 $\eta_1 = \eta_2 = 3$ 采样 $s||e$ 共 4 个 poly; GM 形状锁定为 [4,256] (T-B2 polyvec4)。
4. 行 16-20: polyvec4 NTT 后计算 $\hat{t} = \hat{A} \circ \hat{s} + \hat{e}$; InnerProduct 输出 $P_OUT = 2$, AIV 分片为 1+1。
5. 行 21: 在 compute launch 内融合生成 $ek_pke = \text{ByteEncode}_{12}(\hat{t})||\rho$, 同时写 $dk_pke = \text{ByteEncode}_{12}(\hat{s})$ 。

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 prep	AIV_ONLY, blockDim=2	$\hat{A}$ 共 4 poly 按 2+2 分给两个 AIV; $s e$ 共 4 poly 走 CBD- $\eta = 3$ 写真实 polyvec4 GM。选择 2 AIV 是 D13 k2 探针已验锁定值。
2 compute	MIX, blockDim=1	1 AIC + 2 AIV (S-1 单 cube); polyvec4 NTT; InnerProduct 输出 2 行, AIV0/AIV1 各负责 $\hat{t}[0] / \hat{t}[1]$ 。禁止跨 cube 切 poly。

3.2 GM 几何

对象	锁定几何
$\hat{A}$	[4,256] int32; 4 个真实 poly, 禁止 pad 到 9/16
$s e$	[4,256]; polyvec4, NTT 前后都按真实 4 poly 管理
NTT S0	[8,256] int8, 即 [HI4, L04]; 每个 AIV 持有完整 poly 的 hi+lo, 禁止 limbsplit
Stage2 mat_c	[32,128] int32; Cube 的 M 对齐垫到 16 只是硬件对齐, 不表示假 poly
ByteEncode12	$2 \times 384 = 768\text{B}$; 分别用于 $\hat{t}$ 与 $\hat{s}$; ek 再拼 ρ 得 800B

3.3 同步与流水

Prep 内部使用 UB/GM 分段搬运与必要的 PipeBarrier; 两次 launch 之间由 Host 顺序保证 GM 可见性。Compute 内部遵守 B5/B6/D13 已验模式: AIV 写入平面 Stage1/Stage3 GM 后以 CrossCore flag 与 AIC 同步; AIC 写完 mat_c 后 AIV 等待再做 Stage3 merge、InnerProduct 与 ByteEncode12。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E13 独有的新 AscendC API；下表 API 均已有索引记录，使用约束按 `library /documents/CANN-AscendC算子开发接口参考-查阅索引.md` 执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	63B/UB 张量契约须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运；32B 对齐约束
Duplicate	Memory 矢量计算	√	uint8/int32	UB 清零、pad 与 tail buffer 初始化
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接；CPU 过不能删
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	8Bge2 MMAD；硬件 M 对齐垫不表示假 poly
ShiftRight	矢量算术	√	int32	limb 拆分、Barrett 与 ByteEncode 辅助右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助
Cast	类型转换	√ (受限)	int32/int16/half/float/int8	通过 A2C Cast 链；不得假设 int32->int8 单跳
Gather	矢量搬运/选择	√	int32	仅允许非 NTT S1-S3 段；NTT 三段内禁用
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 尾部等标量缺口

4.1 评估过但未采用

API / 路线	不采用原因
Compares/GatherMas k compact 链	Alg.13 E13 不需要新增 reject compact; 沿用 D13 已 验 Mins/标量 rej 路径
高阶 Matmul<>	继承 D13 手写 MMAD + 平面 mat_c 契约
Gather 用于 NTT S1-S3	参数卡与 ML-KEM NTT 定稿禁止
零垫到 polyvec6/8	S-1/T-B2 锁定禁零垫

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
$G(d 2)$ 与 XOF	shared Keccak/SHAKE + UB 流水	100% 设备	标量 Keccak 核属 本仓 shared
SampleNTT 生成 $\hat{A}[4]$	AIV_ONLY + rej 路径 + DataCopy	100% 设备	2+2 分片; 无零 垫
CBD $\eta = 3$ 生成 $s e[4]$	AIV_ONLY + CBD- $\eta = 3$	100% 设备	输出按 polyvec4 真实行数
polyvec4 NTT S1-S3	MIX AIV/AIC + Mmad	100% 设备	NTT 内无 Gather
InnerProduct $P = 2$	Alg.11 向量路径 + AIV 1+1	100% 设备	整除分片
ByteEncode12 + $ek \rho$	ByteEncode12 向量 + 尾部标量 pack	部分向量	pack 尾部标量缺 口沿用 D13 已验 实现

6 验证计划

6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-keygen-k2
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: `ek_pke.bin` 为 800B, `dk_pke.bin` 为 768B; 若开启 `KEYGEN_VERIFY=1`, 两者均与 golden 对拍 PASS。SIM 通过后检查用例根目录无 `stray core*.dump`、`profile_*log*.toml`。

6.2 可选对照

后续 glue 补 liboqs-512 KAT; E13 本轮门禁为 CPU + SIM_DIRECT=1 sim。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

7 产出物

- exp-fips203-mlkem-pke-keygen-k2-实现方案-customspec.tex
- exp-fips203-mlkem-pke-keygen-k2-实现方案-customspec.pdf
- 后续实现文件须与本 customspec 同目录自包含, 且自研 Python/C/AscendC 写详细中文注释。